Design Review

Submission Detail	Student Entry
GROUP ID	P2-8

Purpose
Document the system requirements, hardware connections, RTOS task structure, behavioural models, assumptions, and AI-supported design reasoning before major implementation. Treat this Week 6 submission as an architecture freeze before full integration and Tribunal 1. The design may change later, but major changes must be justified with engineering evidence.

How to use this template

- This is an architecture-freeze document. The team may improve the design later, but major changes must be justified during Tribunal 1.
- Use AI assistants for ideation, critique, and gap checking. Do not paste AI output directly as the final design.
- Summarise, challenge, correct, and justify AI suggestions using engineering reasoning.
- Keep all artefacts consistent: requirements, pin layout, RTOS task table, task interaction diagram, statecharts, assumptions, and AI evidence.

Team Role and Subsystem Ownership

Identify each buddy's primary subsystem responsibility. The role does not need to match the examples exactly, but every major hardware and software responsibility should have a clear owner.

Buddy	Student ID	Primary subsystem / responsibility
Buddy 1 (Hong Yih)	2501110	WiFi and MQTT communication, including topic and message design, telemetry publishing, command reception and forwarding, heartbeat reporting, and connection recovery.
Buddy 2 (Dalton)	2504003	Motor and encoder integration, PID speed control, distance control and repeatable turns.
Buddy 3 (Edmund)	2501578	IR sensor calibration, line following, barcode decoding and navigation command generation.
Buddy 4 (Khang)	2503012	IMU calibration, motion and hump detection, highest-peak measurement and event reporting.
Buddy 5 (Timothy)	2501530	Servo-mounted ultrasonic scanning, obstacle profiling, avoidance planning and line reacquisition.

Submission Instructions

- Complete every required section.
- Diagrams may be produced using any suitable tool, but labels must remain readable when inserted into this document.
- AI assistants may support drafting. The team remains responsible for technical correctness, consistency, and the ability to explain every decision.
- Use consistent names across the requirements, hardware sketch, task table, task interaction diagram, and statecharts.

1. Requirements

State what the system must do and the important quality or timing constraints. Requirements should be specific, testable, and traceable to the proposed design (you may add more FR and/or NFR as needed).

1.1 Functional Requirements

ID	Requirement	Verification
FR1	The system shall autonomously navigate the predefined course without remote steering or human intervention during the run.	Complete the course without human intervention.
FR2	The system shall continuously detect and follow the track line using the three IR sensors.	Verify that the robot follows straight and curved track sections without losing the line under normal course conditions.
FR3	The system shall detect and decode navigation barcodes and execute the corresponding Left, Right, Straight or U-turn command.	Present each barcode type and verify the corresponding manoeuvre.
FR4	The system shall control both motors using encoder feedback to maintain vehicle speed, travel distance and execute repeatable turns.	Test commanded straight-line distances and turns.
FR5	The system shall use the IMU to monitor vehicle motion, detect tilt, humps and collision events, report turn rate, estimate hump peak height, and retain the highest estimated hump peak height encountered during a run.	Test stationary movement, straight travel, turns, known humps and controlled collision-event conditions; compare reported events and measurements against reference observations.
FR6	The system shall detect obstacles using the ultrasonic sensor, perform coarse scanning followed by fine scanning around detected obstacles, and estimate their position, width and available clearance.	Place obstacles at known positions and dimensions; verify the coarse-to-fine scanning sequence and compare the resulting profiles against reference measurements.
FR7	When an obstacle blocks the line, the system shall determine and execute a collision-free bypass manoeuvre and subsequently reacquire the original line.	Robot successfully bypasses an obstacle and resumes line following without manual intervention.
FR8	The system shall publish robot status and sensor information through WiFi/MQTT, and subscribe to, validate and forward project-defined commands to the Vehicle Controller.	Observe valid MQTT telemetry during a complete run; send valid and invalid test commands and confirm that only valid commands are forwarded.
FR9	The system shall coordinate sensing, navigation, motion, terrain monitoring, obstacle avoidance and telemetry according to the current mission state.	Verify correct state transitions during a complete course run.
FR10	The system shall stop safely on persistent line loss, navigation sensor or encoder failure, motion timeout, or no safe obstacle route, and resume only after recovery.	Trigger each fault; confirm motor stop, fault reporting and controlled recovery.

1.2 Non-Functional Requirements

ID	Requirement	Acceptance Criteria
NFR1	The software shall run on the Raspberry Pi Pico W + Robo Pico, use C and the Pico C SDK, and use the selected µT-Kernel 3.0 RP2040 port.	Application compiles, runs and performs all required functions on the target hardware.
NFR2	Line sensing and motor control shall run at least every 20 ms; encoder interrupts shall capture wheel movement.	Control cycle ≤ 20 ms (50 Hz) with no missed deadlines during normal operation.
NFR3	Obstacle detection and collision-related events shall take priority over navigation and telemetry activities.	Initiate stopping or slowing within 100 ms of confirming an unsafe obstacle condition.
NFR4	The robot shall operate continuously without task failure, deadlock or unintended reset during a complete mission.	Successfully complete 3 consecutive full-course runs without manual intervention or software reset.
NFR5	Loss of WiFi/MQTT connectivity shall not prevent essential autonomous sensing and motion control. The communication subsystem shall support connection recovery and heartbeat reporting.	Disconnect networking during operation and verify autonomous control continues and telemetry reconnects automatically. Connection recovery and heartbeat are explicit project requirements.
NFR6	The implementation shall use CPU time, memory and communication resources efficiently and shall prioritise mission-critical RTOS tasks over non-critical telemetry.	No memory exhaustion, persistent queue overflow or control-task deadline misses during a complete mission. Resource efficiency is an assessed project criterion.
NFR7	Each subsystem shall expose a defined interface and each shared peripheral shall have a designated owner, allowing independent tests and controlled integration.	Each subsystem passes a repeatable standalone test; interface documentation and the task design identify the owner of each shared peripheral.
NFR8	The software shall follow the project repository structure and Barr C Coding Standard and maintain clear separation between RTOS tasks and subsystem modules.	Code review finds no identified violations of the required project structure/coding standard.

2. Hardware Integration Pin Layout

Sketch how sensors, actuators, displays, communication modules, and power connections interface with the Robo Pico. Label pins and interfaces such as GPIO, ADC, PWM, I2C, SPI, and UART.

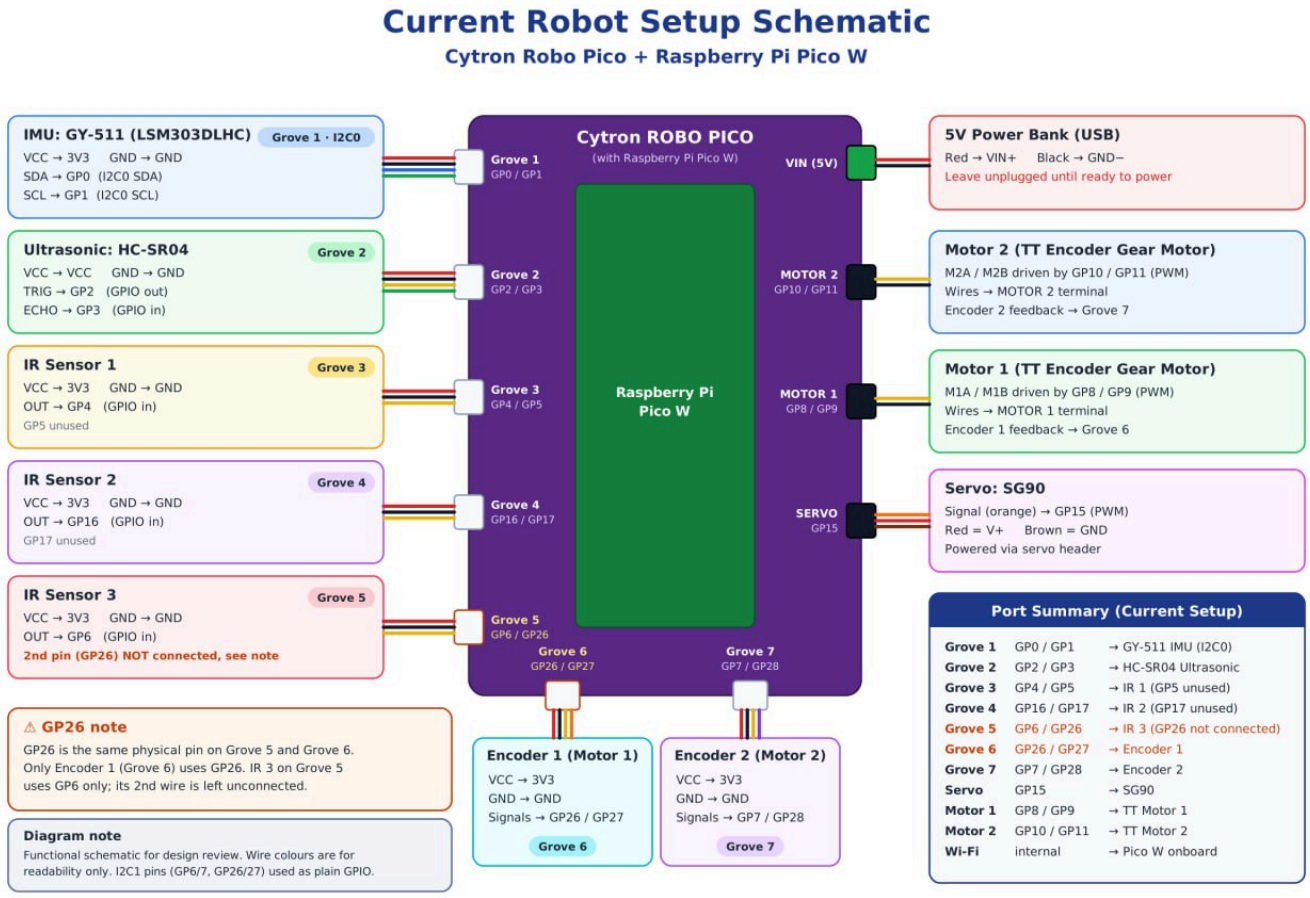


Figure 1. Current robot setup schematic.

Device / Module	Type	Signal Name	Sensor label	Pico GPIO Pin	Interface	Notes / RTOS Task Owner
IMU (GY-511 / LSM303DLHC)	Sensor	IMU_SDA	SDA	GP0	I2C0	Grove 1; IMU Task owns I²C0. Reserve GP0/GP1 for I²C; disable or relocate the RTOS UART0 console, including any UART mirror.
IMU (GY-511 / LSM303DLHC)	Sensor	IMU_SCL	SCL	GP1	I2C0	Clock line. Powered from 3V3.
Ultrasonic sensor (HC-SR04)	Sensor	US_TRIG	TRIG	GP2	GPIO output	Grove 2 signal; Obstacle Scan Task sends the trigger pulse. Power HC-SR04 from 5 V VIN/V+ with common ground, not Grove 3V3.
Ultrasonic sensor (HC-SR04)	Sensor	US_ECHO	ECHO	GP3	GPIO input	Obstacle Scan Task times the echo pulse. Shift the 5 V echo signal to 3.3 V before GP3; do not connect ECHO directly.

Device / Module	Type	Signal Name	Sensor label	Pico GPIO Pin	Interface	Notes / RTOS Task Owner
IR line sensor 1 (IR reflective module)	Sensor	IR1_OUT	OUT	GP4	GPIO input	Grove 3. Line Sensor Task reads it for line position and barcode decoding. GP5 unused.
IR line sensor 2 (IR reflective module)	Sensor	IR2_OUT	OUT	GP16	GPIO input	Grove 4; Line Sensor Task reads GP16. GP17 is unused. Disable or reassign the RTOS demo's default GP16 liveness LED.
IR line sensor 3 (IR reflective module)	Sensor	IR3_OUT	OUT	GP6	GPIO input	Grove 5; Line Sensor Task input on GP6. Grove 5's GP26 contact is not connected at the sensor; GP26 is shared with Grove 6/ENC1_A.
Wheel encoder 1 (encoder module for Motor 1)	Sensor	ENC1_A	Signal (A)	GP26	GPIO input (interrupt)	Grove 6; Motion Control Task owns GP26/GP27. Do not enable the RTOS sample ADC driver on the encoder pins.
Wheel encoder 1 (encoder module for Motor 1)	Sensor	ENC1_B	Signal (B)	GP27	GPIO input (interrupt)	Second channel gives direction.
Wheel encoder 2 (encoder module for Motor 2)	Sensor	ENC2_A	Signal (A)	GP7	GPIO input (interrupt)	Grove 7; Motion Control Task owns GP7/GP28. Keep the GP28 battery-sense jumper open.
Wheel encoder 2 (encoder module for Motor 2)	Sensor	ENC2_B	Signal (B)	GP28	GPIO input (interrupt)	Second channel gives direction.
DC motor 1 (TT gear motor)	Actuator	M1A_PWM	M1A	GP8	PWM	Motion Control Task only. Wired to the MOTOR 1 terminal.
DC motor 1 (TT gear motor)	Actuator	M1B_PWM	M1B	GP9	PWM	Forward when A is high and B is low. Backward is the reverse.
DC motor 2 (TT gear motor)	Actuator	M2A_PWM	M2A	GP10	PWM	Motion Control Task only. Wired to the MOTOR 2 terminal.
DC motor 2 (TT gear motor)	Actuator	M2B_PWM	M2B	GP11	PWM	Forward when A is high and B is low. Backward is the reverse.
Servo motor (SG90)	Actuator	SERVO_PWM	Signal (orange)	GP15	PWM	Servo header. Obstacle Scan Task owns it and sweeps the ultrasonic sensor. Powered from V+ (red).
WiFi (Pico W onboard CYW43)	Communication	WIFI (internal)	N/A	GP23, GP24, GP25, GP29 (internal)	Internal SPI (PIO)	No external wiring. Wireless Comms Task owns the MQTT link. Don't use these GPIOs for anything else.
Power bank (5V USB)	Power	VIN+ / GND	VIN (5V) terminal	N/A	Power input	5 V power bank feeds Robo-Pico VIN/GND, the motors, servo and HC-SR04 VCC. Keep all signal grounds common. Leave unplugged until ready to power.

3. RTOS Task Table

List the proposed RTOS tasks. Priorities are preliminary at Week 6 and must be justified.

Task	Purpose	Inputs	Outputs	Trigger	Priority
Vehicle Controller (Dalton)	Coordinate mission state and safe stop	Line/obstacle/fault events; valid commands	Motion commands; stop/fault state	Event-driven; watchdog	Highest
Motion Control (Dalton)	Control speed, distance and turns via encoder PID	Encoder counts; motion commands	Motor PWM; odometry; stall fault	Periodic, ≤20 ms	Very high
Obstacle Scan (Timothy)	Sweep servo; profile obstacles; plan bypass	HC-SR04 echo; mission mode	Hazard event; clearance; bypass plan	Periodic; non-blocking	High
Line Sensor (Edmund)	Track line and decode barcodes	IR1–IR3; calibration	Line position; barcode; line-loss fault	Periodic, ≤20 ms	Medium
IMU/Terrain (Khang)	Detect humps and retain highest peak	GY-511 I²C0 readings	Terrain events; highest peak	Periodic; rate to validate	Low
Wireless Comms (Hong Yih)	Maintain WiFi/MQTT, telemetry and commands	System status; broker messages	Telemetry; validated commands	Event/retry; heartbeat	Lowest

Priority and ownership justification:

The Vehicle Controller has Highest priority so faults and obstacle events can trigger a safe stop promptly. Motion Control has Very high priority to keep its motor and encoder control cycle within 20 ms. Obstacle Scan has High priority to report hazards before routine navigation and telemetry; it waits for measurements without busy-waiting. Line Sensor has Medium priority and still runs at least every 20 ms to support line following. IMU/Terrain has Low priority because hump and peak processing can follow the immediate control work. Wireless Comms has Lowest priority so telemetry and connection retries do not delay autonomous driving. Dalton owns the controller and motion tasks because he owns the motors and encoders. Timothy owns obstacle scanning, Edmund line sensing, Khang IMU sensing and Hong Yih communications. Only Motion Control writes motor PWM; encoder ISRs capture pulses for it to process.

4. Task Interaction Diagram

Show RTOS tasks, queues, semaphores, mutexes, event groups, notifications, and data flow. Clearly distinguish software tasks from hardware devices and interrupt service routines.

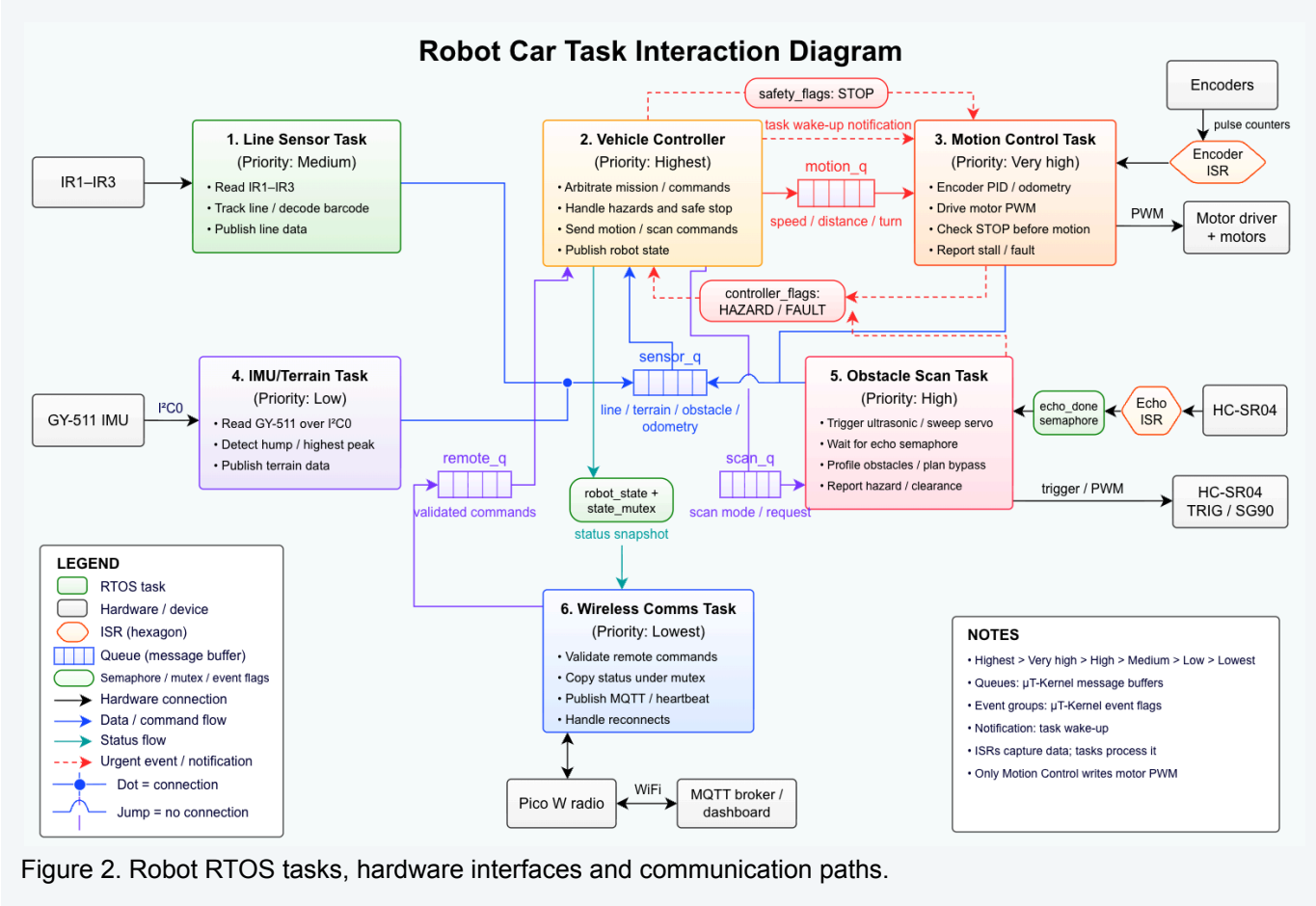


Figure 2. Robot RTOS tasks, hardware interfaces and communication paths.

Brief explanation of the main communication paths:

Data and commands. Line Sensor, Obstacle Scan, IMU/Terrain and Motion Control send timestamped readings or results through sensor_q to the Vehicle Controller. Wireless Comms validates remote commands before placing them in remote_q. The controller sends speed/distance/turn commands through motion_q to Motion Control, and scan requests through scan_q to Obstacle Scan. These queues use µT-Kernel message buffers.

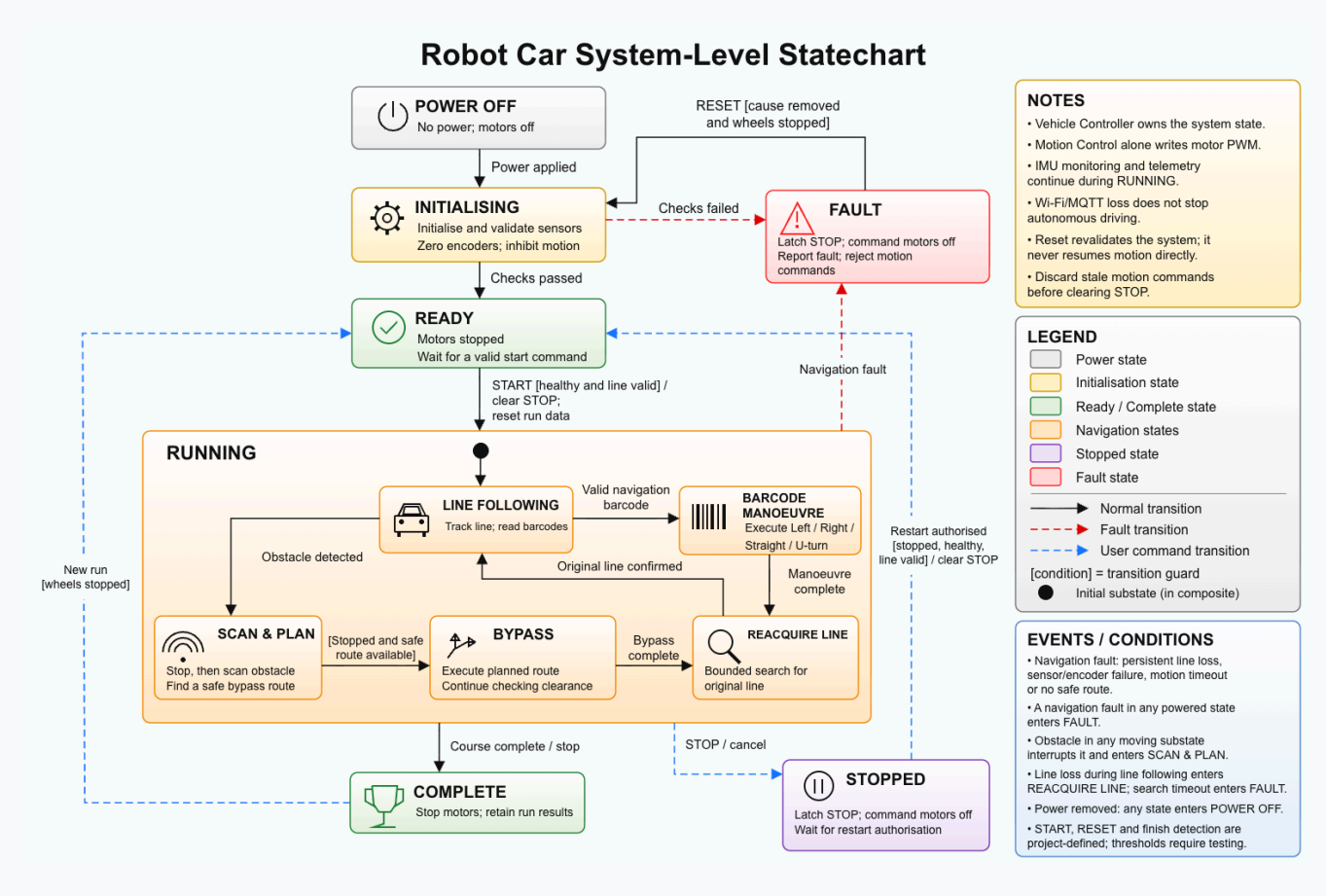
Hardware and interrupts. Obstacle Scan alone controls the ultrasonic trigger and servo. The Echo ISR records pulse timing and signals echo_done; Obstacle Scan waits on this semaphore with a timeout and calculates distance. Encoder ISRs only update pulse counters; Motion Control takes a consistent snapshot using a short ISR-safe critical section and performs PID processing. Only Motion Control writes motor PWM. ISRs never wait on a mutex.

Safety and notifications. Obstacle Scan sets HAZARD and Motion Control sets FAULT in controller_flags to alert the Vehicle Controller. The controller checks both input queues at each event or watchdog timeout, at most 20 ms apart. To stop safely, it sets the latched STOP bit in safety_flags and sends Motion Control a task wake-up notification. Motion Control checks STOP before executing commands. Event groups use µT-Kernel event flags; notifications use task wake-up. The controller clears STOP only after authorising recovery and discarding stale motion commands.

Telemetry and shared state. The Vehicle Controller updates robot_state under state_mutex. Wireless Comms takes the same mutex only long enough to copy a consistent snapshot, releases it, then publishes MQTT telemetry. The mutex uses priority inheritance and short critical sections so networking does not hold up control.

5. System-Level Statechart

Model the operating states of the entire system. Label transition events or conditions. Include relevant fault, stop, or recovery behaviour.



Explain the main states and any important assumptions:

1. Start-up and control ownership. The Vehicle Controller owns the system state, while Motion Control alone writes motor PWM. Applying power moves POWER OFF to INITIALISING, where sensors are checked and encoder counts are reset with motion inhibited. Successful checks lead to READY. START is accepted only when the system is healthy and the line is valid; stale motion commands are discarded before clearing STOP and resetting run data.

2. Navigation within RUNNING. LINE FOLLOWING is the initial substate. A valid navigation barcode selects BARCODE MANOEUVRE for a left, right, straight or U-turn command. An obstacle interrupts a moving substate and enters SCAN & PLAN: the robot stops, scans and selects a safe route before BYPASS. Both manoeuvres lead to REACQUIRE LINE, which confirms the original line before returning to LINE FOLLOWING. Line loss also starts this bounded search; a search timeout or no safe bypass route causes FAULT.

3. Stop and restart. STOP or cancel during RUNNING enters STOPPED, latches STOP and commands the motors off. Restart requires authorisation, stationary wheels, healthy subsystems and a valid line. The controller discards stale motion commands, clears STOP and returns to READY, where a new START is required.

4. Fault recovery. Failed initial checks, persistent line loss, sensor or encoder failure, or a motion timeout enter FAULT. STOP remains latched, the motors are commanded off, and further motion commands are rejected. RESET is accepted only after the cause is removed and the wheels have stopped. It returns the system to INITIALISING for revalidation; movement still requires successful checks and a new START.

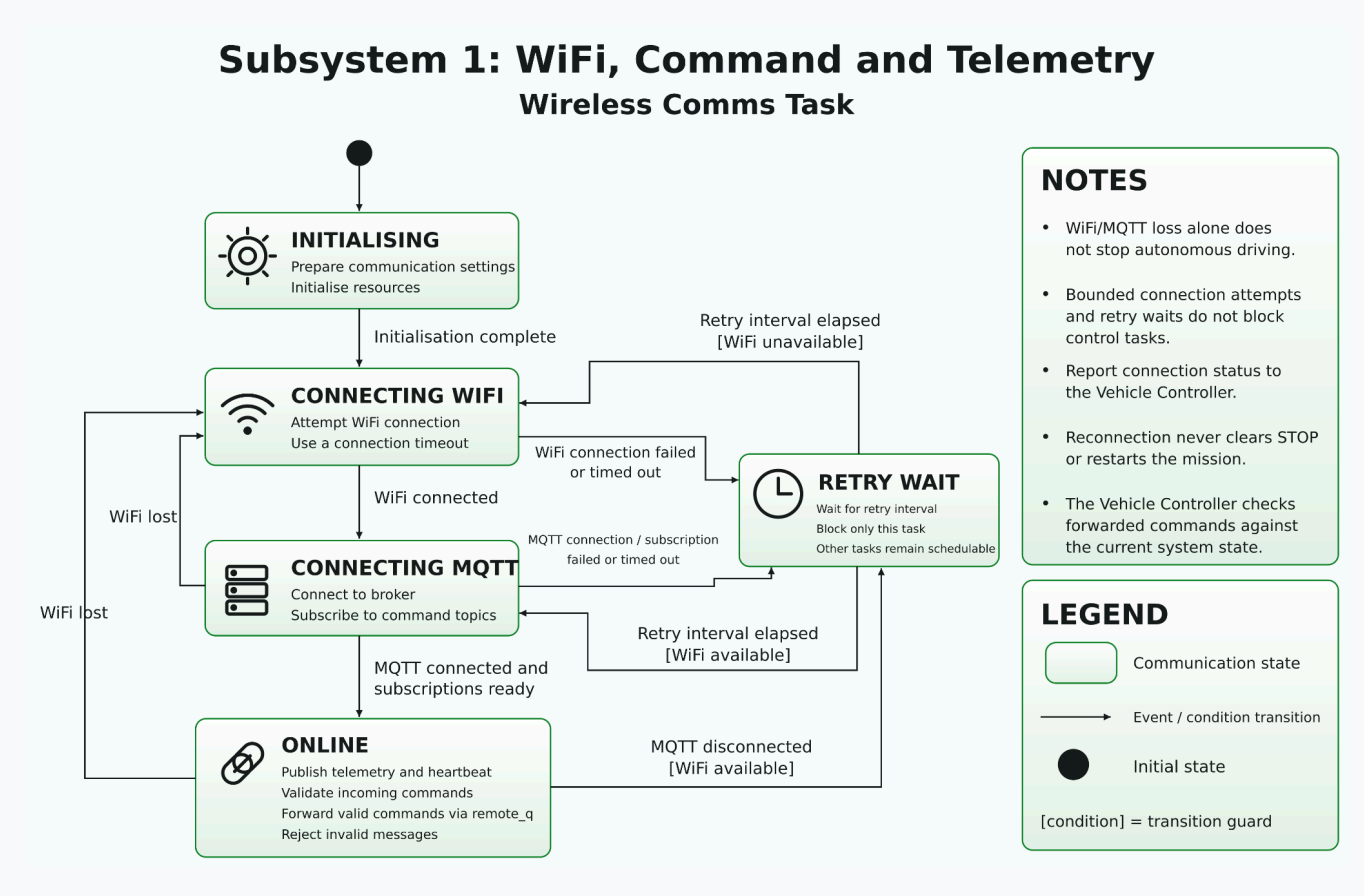
5. Completion and power loss. Course completion enters COMPLETE, stops the motors and retains run results. A new-run request returns to READY once the wheels have stopped. Removing power from any state leads to POWER OFF.

6. Assumptions. IMU monitoring and telemetry continue during RUNNING. Wi-Fi/MQTT loss alone does not stop autonomous driving. START, RESET and finish detection follow project-defined conditions; obstacle clearance, line validity and timeout thresholds require testing.

6. Subsystem Statecharts

6.1 Subsystem Statechart 1

Field	Student Entry
Subsystem name	WiFi Communication, Command and Telemetry
Related RTOS task(s)	Wireless Comms Task
Purpose	Establish and maintain WiFi and MQTT connections, publish robot status and telemetry, receive remote commands, and transmit periodic heartbeat messages. Forward valid commands to the Vehicle Controller for processing.
Reason a statechart is needed	The subsystem behaves differently during initialisation, WiFi connection, MQTT connection, normal communication, and connection recovery. A statechart defines the events that cause transitions and explains how communication resumes after a failure.



Brief explanation of states, events, and fault handling:

The subsystem begins in Initialising, where communication settings and resources are prepared. It then connects to WiFi before connecting to the MQTT broker and subscribing to the command topics.

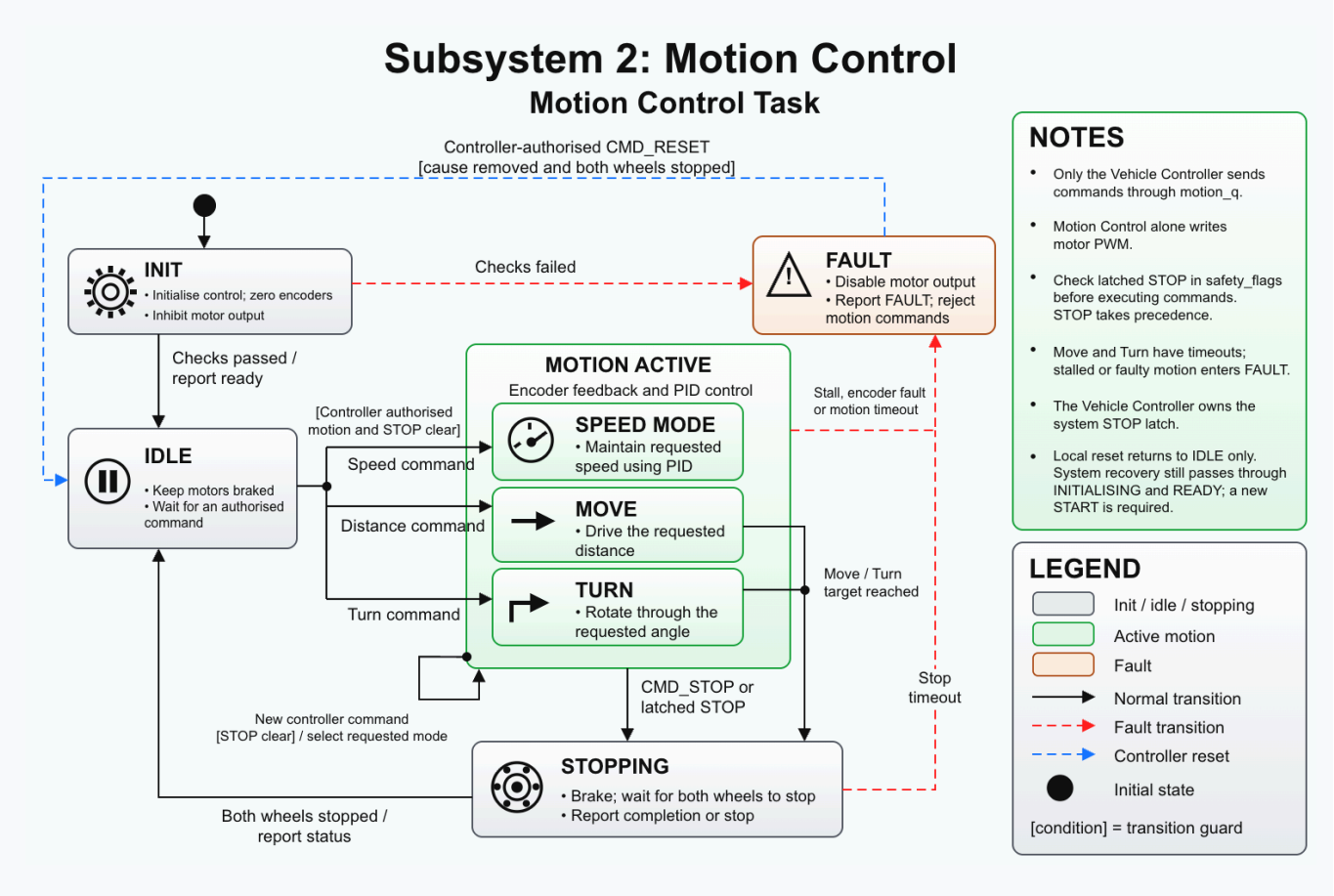
In the Online state, the subsystem publishes robot telemetry and periodic heartbeat messages. Incoming commands are checked for a valid format and supported command type before being forwarded to the Vehicle Controller. Invalid messages are rejected.

If WiFi is lost, the subsystem returns to ConnectingWiFi. If the MQTT connection fails while WiFi remains available, it waits for a configured retry interval before attempting to reconnect and restore its subscriptions.

Connection attempts use timeouts so communication failures do not block other robot tasks. WiFi/MQTT loss alone does not stop autonomous sensing or motion control. The subsystem reports connection status to the Vehicle Controller and retries the connection independently. Reconnection does not clear STOP or restart the mission; received commands still require controller validation against the current system state.

6.2 Subsystem Statechart 2

Field	Student Entry
Subsystem name	Motion Control (motors + wheel encoders)
Related RTOS task(s)	Motion Control Task. It reads encoder counts captured by GPIO ISRs and accepts motion commands only from the Vehicle Controller through motion_q. It checks the latched STOP in safety_flags before executing commands and alone writes motor PWM.
Purpose	Drive the two DC motors so the car moves at a controlled speed, travels a set distance, keeps straight and turns by a set angle, using encoder feedback and PID control.
Reason a statechart is needed	The motors behave differently when holding a speed, moving a distance, turning or braking. The Vehicle Controller coordinates requests and sends the resulting motion commands. A statechart defines command handling, STOP precedence and fault recovery so a new command cannot bypass a stop or fault.



Brief explanation of states, events, and fault handling:

Init: Initialises motion control and resets encoder counts with motor output inhibited. Successful checks enter Idle and report readiness to the Vehicle Controller. Failed setup reports a fault and prevents motion, allowing the controller to keep the system in FAULT.

Idle: The motors stay braked while awaiting a valid motion command from the Vehicle Controller. Speed Mode, Move or Turn may begin only when the controller has cleared the latched STOP and authorised motion. Commands received while STOP is set are not executed. Returning to Idle does not restart the mission.

Speed Mode: Runs the PID speed control loop to maintain the requested speed. CMD_STOP or the latched STOP takes precedence over motion commands and enters Stopping. A new controller command may replace the current one only while STOP is cleared. A stall, encoder fault or other motion-control fault enters Fault.

Move: Drives the requested distance using encoder feedback and PID control. Reaching the target, CMD_STOP or the latched STOP enters Stopping. A new controller command may replace the current one only while STOP is cleared. A timeout, motor stall or encoder fault enters Fault.

Turn: Rotates the car through the requested angle. Reaching the target, CMD_STOP or the latched STOP enters Stopping. A new controller command may replace the current one only while STOP is cleared. A timeout, motor stall or encoder fault enters Fault.

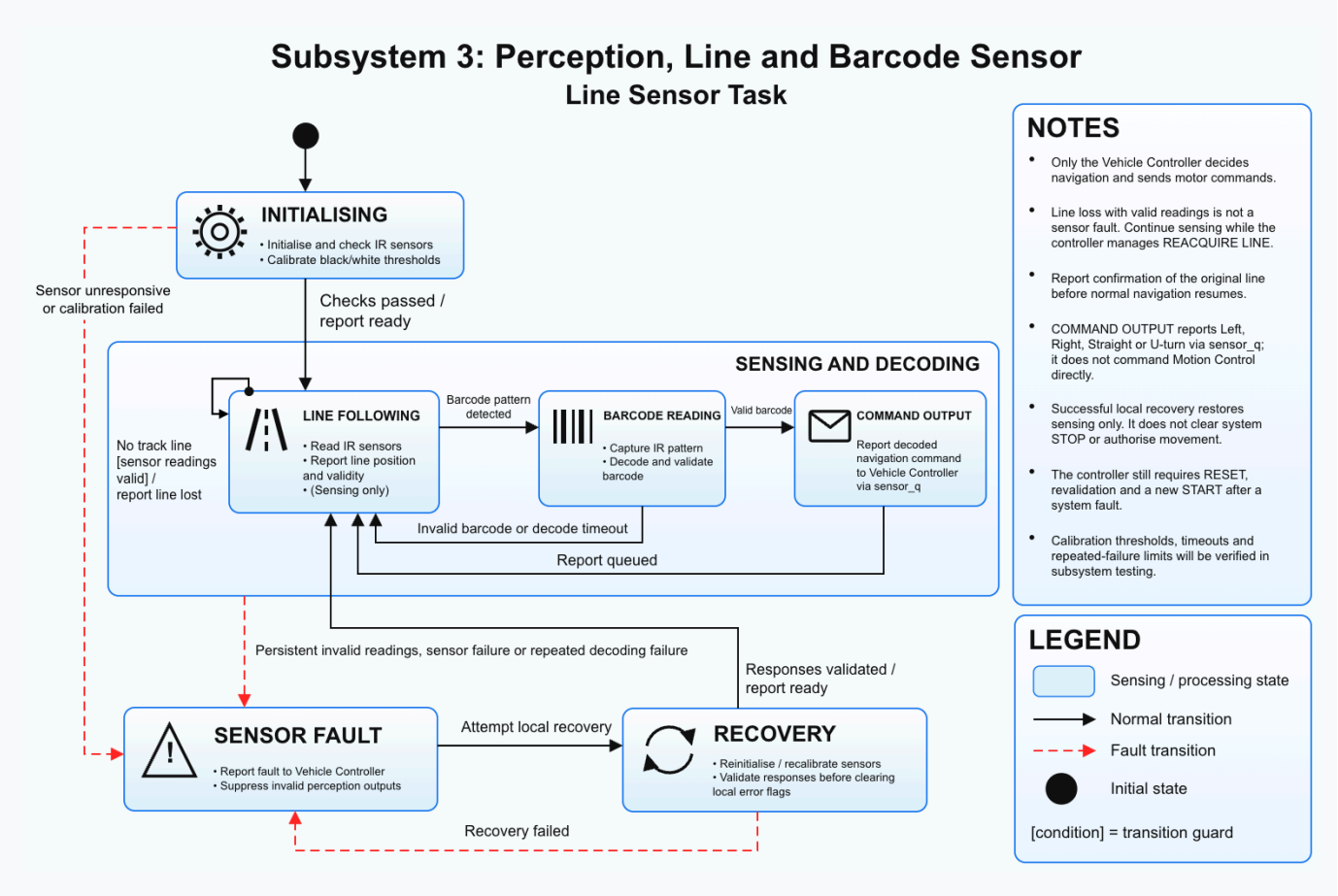
Stopping: Applies braking and waits until both wheels have stopped, then reports the completed motion or stop to the Vehicle Controller and returns to Idle. The system STOP remains under controller ownership. Failure to stop within the allowed time enters Fault.

Fault: Disables motor output, signals FAULT to the Vehicle Controller and rejects motion commands. The controller latches the system STOP. Return to Idle requires a controller-authorised CMD_RESET, the fault cause

removed and both wheels stopped. This resets only the local motion fault: system recovery still passes through INITIALISING and READY, with a new START required before movement.

6.3 Subsystem Statechart 3

Field	Student Entry
Subsystem name	Perception / Line & Barcode Sensor Subsystem
Related RTOS task(s)	Line Sensor Task (IR sampling, line detection and barcode decoding). Reports perception data and events to the Vehicle Controller through sensor_q; it does not send commands directly to Motion Control.
Purpose	Acquire IR readings, determine line position and validity, detect and decode navigation barcodes, and report line-loss, line-reacquisition and sensor-fault events to the Vehicle Controller. The controller decides the navigation response and issues all motor commands.
Reason a statechart is needed	To clearly define the different sensing and detection modes (line following, barcode detection), handle transitions between them, and manage fault conditions such as sensor failure or invalid barcode.



Brief explanation of states, events, and fault handling:

Initialization. The subsystem starts by initialising the IR sensors, calibrating the black/white thresholds, checking the sensor responses, and setting the default parameters. Successful initialisation is reported to the Vehicle Controller and enters Line Following for sensing only; movement remains subject to the system START and STOP conditions. If a sensor does not respond or calibration fails, the subsystem enters **Sensor Fault** instead of continuing normal operation.

Line Following. The Line Sensor Task reads the IR sensors and reports line position and validity to the Vehicle Controller. Valid readings with no track line generate a line-loss event; sensing continues while the controller manages REACQUIRE LINE. Confirmation of the original line is reported before normal navigation resumes. A barcode pattern enters Barcode Reading. Persistent invalid readings or an unresponsive sensor enter Sensor Fault; these are distinct from simply losing the track line.

Barcode Reading. When a possible barcode is detected, the subsystem captures the IR pattern, decodes the barcode, and validates the corresponding navigation command. If a valid barcode is obtained, the subsystem transitions to **Command Output**. If the barcode is invalid or a timeout occurs, the subsystem returns to **Line Following** and continues normal track detection. Repeated barcode decoding failures may cause the subsystem to enter **Sensor Fault**.

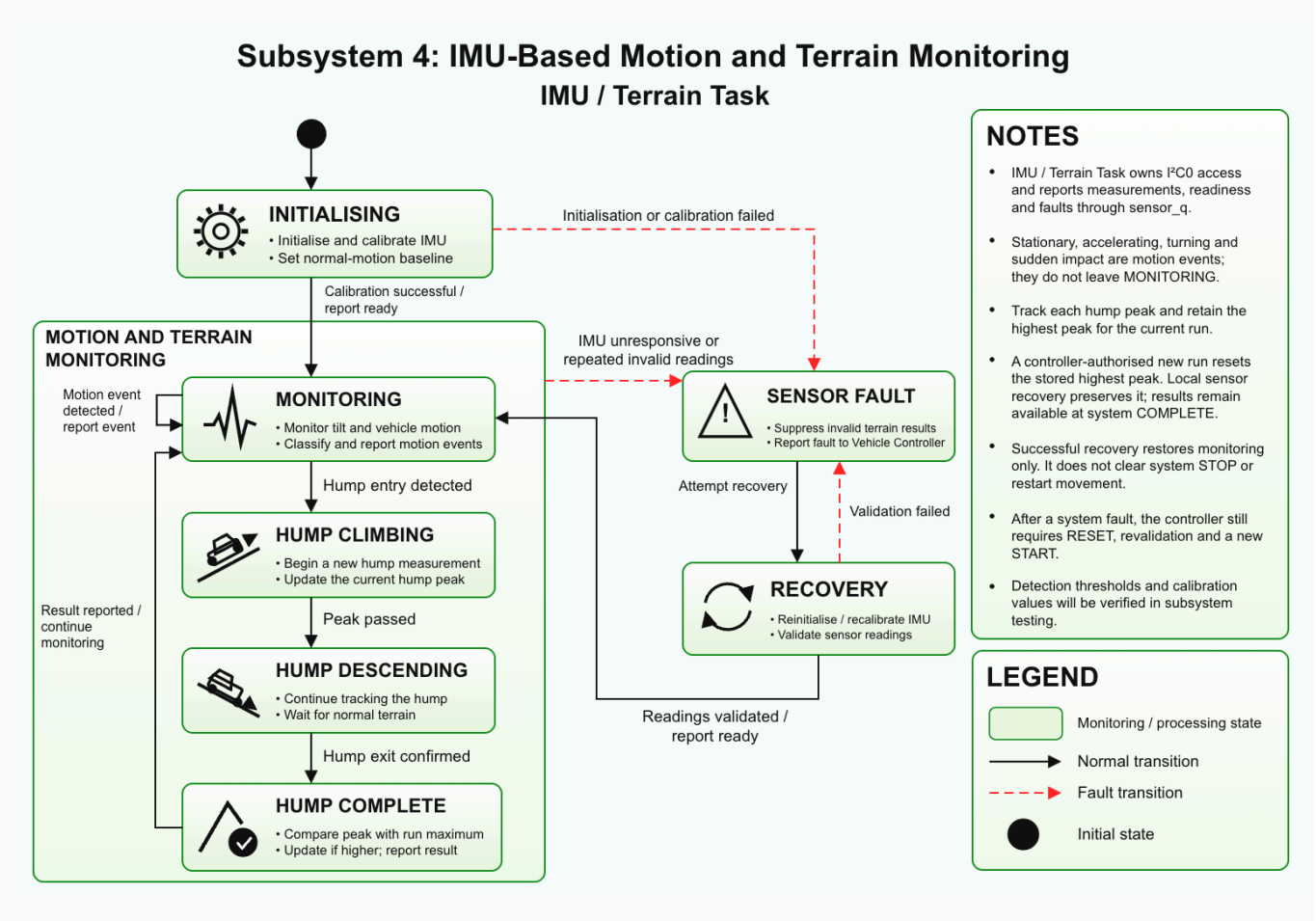
Command Output. Once a barcode has been successfully decoded, the subsystem sends the corresponding navigation command, such as **Left, Right, Straight, or U-turn**, to the vehicle controller. After the command has been sent, the subsystem returns to **Line Following** so that normal track monitoring can continue. If a valid command cannot be generated because of invalid perception data, the error is handled through **Sensor Fault**.

Sensor Fault and Recovery. Persistent invalid readings, loss of sensor response, calibration failure or repeated barcode decoding errors enter Sensor Fault. The subsystem reports the fault to the Vehicle Controller and suppresses invalid perception outputs. Recovery reinitialises the sensors, recalibrates if required and validates their responses before clearing local error flags. Successful recovery resumes sensing in Line Following and reports readiness; it does not clear the system STOP or authorise movement. The controller still requires RESET, revalidation and a new START. Failed recovery returns to Sensor Fault.

Design and traceability note: This statechart supports the perception subsystem responsible for line tracking and barcode detection. Exact calibration thresholds, timeout values, and the number of repeated decoding failures required to trigger a fault will be confirmed through subsystem testing and calibration before full system integration.

6.4 Subsystem Statechart 4

Field	Student Entry
Subsystem name	IMU-Based Motion and Terrain Monitoring
Related RTOS task(s)	IMU/Terrain Task. Owns IMU access over I²C0, processes motion and terrain readings, and reports measurements, readiness and fault events to the Vehicle Controller through sensor_q.
Purpose	Monitor the vehicle's motion and terrain using the IMU. The subsystem detects tilt and humps, measures each hump peak, keeps the highest peak from the current run, classifies motion such as stationary, accelerating and turning, and reports relevant events to the vehicle controller and telemetry subsystem.
Reason a statechart is needed	The IMU readings depend on the current condition and what happened before. For example, a hump has to be recognised as a sequence of climbing, passing the peak, descending and returning to normal terrain. The statechart also makes fault and recovery behaviour clear so invalid sensor data is not treated as a real event.



Brief explanation of states, events, and fault handling:

Initialization. The subsystem starts by initialising and calibrating the IMU, then sets the baseline values used for normal motion and terrain monitoring. Successful initialisation is reported to the Vehicle Controller. If initialisation or calibration fails, it enters Sensor Fault and reports the failure instead of using unreliable data.

Monitoring. During normal operation, the IMU/Terrain Task continuously processes sensor readings to monitor tilt and vehicle motion. Conditions such as stationary, accelerating, turning and sudden impact are classified as motion events and reported while the subsystem remains in Monitoring.

Hump Climbing and Hump Descending. When the readings indicate the start of a hump, the subsystem begins a new measurement and enters **Hump Climbing**. The current peak is updated while the vehicle climbs. After the peak is passed, it enters **Hump Descending** and continues monitoring until the vehicle returns to normal terrain.

Hump Complete. Once the hump ends, its peak is compared with the highest peak recorded for the run, updating the maximum only if it is higher. The result is reported to the Vehicle Controller before returning to Monitoring. Run results are retained at COMPLETE. A controller-authorised new run resets the stored highest peak; local sensor recovery does not erase it.

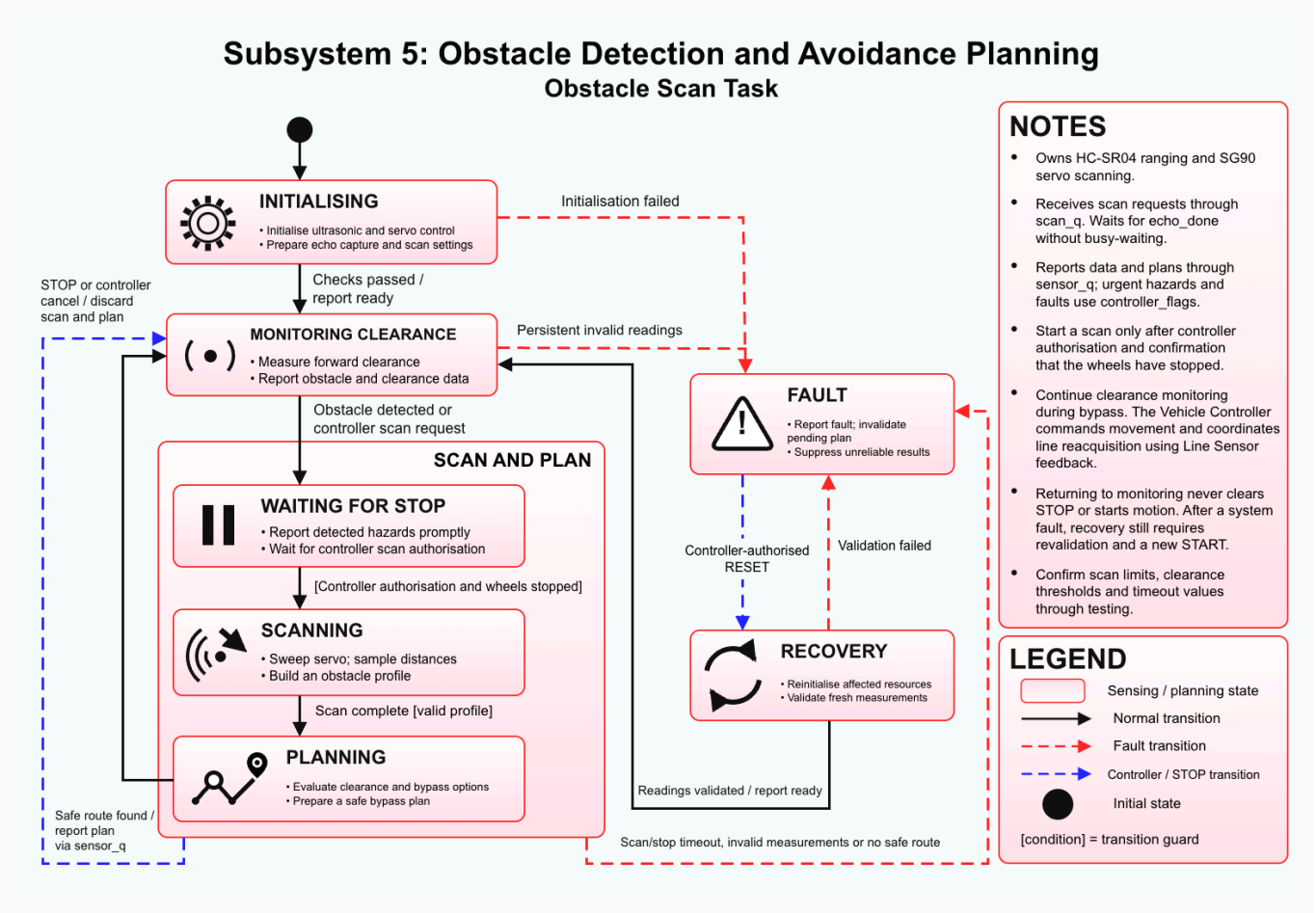
Sensor Fault and Recovery. If the IMU stops responding or repeatedly gives invalid readings, the subsystem enters Sensor Fault, suppresses invalid terrain results and reports the fault to the Vehicle Controller for system-level FAULT handling. Recovery reinitialises and, if needed, recalibrates the sensor, then validates its readings. Success resumes Monitoring and reports readiness, but does not clear the system STOP or restart motion. The controller still requires RESET, revalidation and a new START. Failed recovery remains faulted.

Design and traceability note: Exact detection thresholds and calibration values will be confirmed through subsystem testing. This design links FR5 to the IMU / Terrain Monitoring Task and Subsystem Statechart 4.

Assumptions A7 and A8 cover IMU stability during movement and distinguishing hump readings from normal vibration. Measurements and fault/readiness reports use sensor_q to the Vehicle Controller; detailed message payloads will be confirmed during integration.

6.5 Subsystem Statechart 5

Field	Student Entry
Subsystem name	Obstacle Detection and Avoidance Planning
Related RTOS task(s)	Obstacle Scan Task. Controls HC-SR04 ranging and SG90 scanning, receives requests through scan_q, and waits for echo_done from the Echo ISR. Reports obstacle data and bypass plans through sensor_q, with urgent hazards and faults via controller_flags.
Purpose	Monitor clearance, detect obstacles and scan their surroundings after the vehicle has stopped. Build an obstacle profile and propose a safe bypass route. Report clearance during bypass; the Vehicle Controller commands Motion Control and coordinates line reacquisition using Line Sensor feedback.
Reason a statechart is needed	The subsystem changes behaviour between clearance monitoring, scanning, planning and fault recovery. A statechart defines when scans may start, how measurements and plans are reported, and how timeouts, invalid readings or no safe route are handled without bypassing system STOP or recovery controls.



Brief explanation of states, events, and fault handling:

INITIALISING prepares the HC-SR04, servo and echo capture. Successful checks lead to MONITORING CLEARANCE, which reports forward clearance and obstacles. An obstacle or controller scan request enters WAITING FOR STOP. SCANNING begins only when the controller authorises it and the wheels have stopped. A complete, valid obstacle profile leads to PLANNING; a safe bypass plan is reported through sensor_q before returning to monitoring. The Vehicle Controller commands the bypass and coordinates line reacquisition. Clearance monitoring continues during bypass.

STOP or controller cancellation discards the scan and pending plan, then returns to monitoring without clearing STOP or starting movement. Urgent hazards and faults are reported through controller_flags.

Initialisation failure, persistent invalid readings, scan/stop timeout or no safe route leads to FAULT, which invalidates the pending plan and suppresses unreliable results. A controller-authorised RESET enters RECOVERY to reinitialise resources and validate fresh readings. Successful validation reports readiness and restores monitoring; failed validation returns to FAULT. After a system fault, movement still requires system revalidation and a new START.

7. Assumptions

List assumptions that the design depends on. Each assumption must state the risk if it is wrong and how the team will validate it before full integration. Have at least 1 assumption per buddy. Use measurable pass criteria from the requirements; agree any remaining tolerances before testing. Keep assumptions Open until supported by test evidence.

ID	Assumption	Risk if wrong	Validation method	Owner	Status
A1	The WiFi connection will remain sufficiently stable throughout the operating area.	Telemetry or commands may be delayed or lost during operation.	Test communication across the course; record losses and delays. Deliberately disconnect and reconnect Wi-Fi; verify controller loss handling and that reconnection does not restart movement.	Buddy 1	Open
A2	Telemetry and reconnection processing can run alongside all other tasks without compromising vehicle-control timing or STOP response.	Communication processing may delay navigation, sensing, or motor-control tasks.	Run all tasks together, including scanning, telemetry and Wi-Fi reconnection. Measure control-task timing and STOP response against the agreed requirements.	Buddy 1	Open
A3	Encoder feedback and motion control can maintain straight-line travel within the required path accuracy.	Inaccurate feedback, wheel slip or unequal motor response may cause drift from the intended path.	Repeat straight-line runs at representative speeds, loads and battery levels. Compare measured path deviation and encoder counts with actual travel.	Buddy 2	Open
A4	Encoder feedback and motion control can execute turns with sufficient repeatability for navigation.	The vehicle may under-turn or over-turn and fail to correctly enter the next section of the course.	Repeat Left, Right and U-turns at representative speeds, loads and battery levels. Measure final heading and position against agreed tolerances.	Buddy 2	Open
A5	The three IR sensors can reliably distinguish the track line from the surrounding surface.	The vehicle may lose the line or make incorrect steering corrections.	Calibrate the sensors, then repeat straight and curved line-following tests at the intended operating speed under expected lighting and course conditions.	Buddy 3	Open
A6	The IR sensors can reliably distinguish barcode bars and spaces at the intended operating speed under expected lighting and course conditions.	Barcodes may be decoded incorrectly, causing the wrong navigation manoeuvre to be executed.	Drive over each barcode type repeatedly at the intended operating speed and under expected lighting. Compare each decoded command with the known barcode.	Buddy 3	Open
A7	The orientation and terrain sensing module will provide sufficiently stable measurements	Vehicle vibration or sensor noise may produce false motion or terrain detections.	Record sensor readings while stationary, moving on a flat surface, turning, and travelling over known terrain features.	Buddy 4	Open

	while the vehicle is moving.				
A8	Hump-related sensor readings can be distinguished from normal vehicle vibration and movement.	Normal vibration may be incorrectly classified as a hump, or an actual hump may not be detected.	Drive over flat surfaces and known humps repeatedly and compare the recorded sensor measurements to determine suitable detection thresholds.	Buddy 4	Open
A9	Ultrasonic ranging can detect course obstacles accurately and early enough for the vehicle to stop with adequate clearance.	Incorrect distance measurements may cause late stopping, unnecessary avoidance, or collision.	Test known distances with representative obstacle sizes, surfaces and angles, including missing echoes. At operating speed, measure detection and stopping distance against the required clearance.	Buddy 5	Open
A10	The servo can position and settle the ultrasonic sensor consistently enough to build a repeatable obstacle profile.	Incorrect scan angles may result in an inaccurate estimate of obstacle position, width, or available clearance.	Repeat scans at known angles and obstacle positions. Establish settling time before sampling; compare scan angles, distance readings and profiles across repeated scans.	Buddy 5	Open
A11	The power supply remains stable during simultaneous motor, servo and Wi-Fi activity.	Supply dips may reset the controller, corrupt readings or interrupt control.	Run motors, servo scanning and Wi-Fi together at representative battery levels. Check supply voltage, resets and sensor behaviour during starts and turns.	Buddy 2	Open
A12	The scan profile supports a bypass with clearance for the whole vehicle and subsequent line reacquisition.	A route may appear clear to the sensor but clip an obstacle or fail to return to the line.	Test representative obstacle layouts using the vehicle dimensions and required clearance. Verify bypass clearance and successful line reacquisition with Motion Control and Line Sensor feedback.	Buddy 5	Open

8. AI-Supported Design Evidence

Document how AI was used to support the design. The aim is not to show that AI produced perfect answers, but to show that the team used AI critically, corrected weak suggestions, and made final engineering decisions.

8.1 Pre-Submission AI Ideation

Buddy	Example prompt/discussion	Useful AI ideas	Weak or unrealistic AI ideas	Team decision
Buddy 1	Review wireless command handling and telemetry under an RTOS.	Separate communication from vehicle control and pass commands through a queue.	Allowing reconnection to resume the previous movement command.	The team kept queued command forwarding to the Vehicle Controller and rejected automatic movement restart after reconnection.
Buddy 2	Review encoder-based motion control and task priorities.	Give motion control clear ownership of motor outputs and prioritise STOP handling.	Allowing several sensing tasks to write motor commands directly.	The team kept exclusive motor ownership by Motion Control and prioritised STOP handling, rejecting direct motor commands from sensing tasks.
Buddy 3	Review line tracking and barcode decoding using three IR sensors.	Separate line observations, barcode decoding and navigation decisions.	Treating every loss of the line as a sensor failure.	The team kept separate sensing, decoding and navigation decisions, but rejected treating all line loss as sensor failure. The controller coordinates line reacquisition.
Buddy 4	Review IMU-based motion classification and hump measurement.	Use filtering and distinct hump phases to organise measurements.	Treating a single abnormal sample as a confirmed terrain event.	The team modified the hump detection approach to use filtered readings and distinct hump phases, with thresholds to be validated through repeated tests, rather than confirming events from one abnormal sample.
Buddy 5	Review ultrasonic scanning and obstacle-bypass planning.	Scan after the vehicle stops, build an obstacle profile and report a bypass plan.	Assuming a clear ultrasonic reading guarantees clearance for the whole vehicle.	The team kept scanning after stopping and modified route assessment to include vehicle dimensions and clearance. It rejected treating one clear ultrasonic reading as proof of a safe route.

8.2 Team-Reviewed Architecture Freeze

Initial idea	Decision to confirm	Engineering reason	Affected artefact
Let individual subsystems initiate movement independently.	Modified: centralise coordination in the Vehicle Controller.	Prevent conflicting commands and maintain consistent STOP behaviour.	Task table, interaction diagram, statecharts
Use continuous polling for ultrasonic echo completion.	Modified: use ISR signalling and bounded waiting.	Avoid unnecessary CPU use and handle missing echoes explicitly.	Interaction diagram, Subsystem 5
Resume operation immediately after reconnection or sensor recovery.	Rejected: retain controller-managed recovery and restart conditions.	Local recovery does not establish that the entire vehicle is ready to move.	System and subsystem statecharts
Give all tasks equal priority.	Modified: prioritise control and urgent stop handling over telemetry.	Reduce interference with time-critical vehicle behaviour.	Task priorities, A2
Execute a bypass as soon as a route is calculated.	Modified: report the plan for controller-coordinated execution.	Keep planning, movement and line reacquisition under defined ownership.	Subsystems 2, 3 and 5; A12

8.3 Final AI-Vetted Gap Check

After completing the design, ask AI to act as a strict embedded systems reviewer. The review should identify missing requirements, unclear interfaces, untested assumptions, weak mitigations, timing concerns, task conflicts, or resource contention.

Use one of these Team Response options: **Fixed**, **Partially fixed**, **Accepted as risk**, **Deferred**, **Rejected with reason**, or **Needs validation**. The action and evidence columns must justify the selected response. **AI reviewer prompt used**	Act as a strict embedded systems reviewer for our autonomous robot car using Robo Pico, Raspberry Pi Pico W and the selected µT-Kernel port. Review requirements, hardware assignments, RTOS tasks, communication paths, statecharts, assumptions and traceability. Identify conflicting ownership, unsafe restart behaviour, timing risks, incomplete interfaces and untested assumptions. Distinguish documented design decisions from validated behaviour. List gaps and supporting section references without rewriting the design.

AI-identified gap	Team response	Action taken	Evidence or reason
Independent subsystem commands or recovery could cause conflicting movement or an unintended restart.	Fixed	The team assigned mission coordination and STOP clearance to the Vehicle Controller and exclusive motor ownership to Motion Control. It defined revalidation and a new START as prerequisites for restarting after a system fault.	Sections 3–6 define ownership and recovery consistently. This resolves the design ambiguity; implementation still requires testing.
Task priorities alone do not demonstrate that control deadlines and obstacle-response targets will be met.	Needs validation	The team defined control periods, response targets, bounded echo waiting and short shared-state access. It scheduled combined-load timing validation for integration testing.	NFR2–NFR3, Section 3 and A2 require verification under scanning, telemetry and reconnection load.
Message formats, queue capacities and behaviour when data becomes stale or queues fill are not fully specified.	Partially fixed	The team defined the communication mechanisms and their producers and consumers. It deferred final message formats, queue capacities, freshness limits and overflow handling to integration.	Section 4 defines communication paths; Section 9 identifies the remaining interface work.
Hardware protection and GPIO reservations are documented but not yet demonstrated on the assembled robot.	Needs validation	The team documented ultrasonic power and echo-level requirements, shared-pin restrictions and RTOS configuration conflicts. It identified physical wiring and power checks for validation before integration.	Section 2 specifies these constraints; A11 covers supply stability. The existing schematic does not establish that every correction is physically implemented.
The expanded sensing requirements are not fully covered by the current descriptions and verification steps.	Partially fixed	The team expanded FR5 and FR6 to cover the required sensing behaviours. Detailed acceptance criteria and the corresponding task and scanning descriptions remained outstanding at submission.	FR5 verification currently focuses on humps and maximum peak reporting. Subsystem 5 describes scanning without explicitly explaining its coarse and fine stages.
The requirements mapping contained an outdated FR1 reference.	Fixed	The team corrected Section 9 to map FR1 to autonomous course navigation through the Vehicle Controller, Line Sensor, Motion Control and Obstacle Scan. It removed the incorrect platform mapping from FR1.	Sections 1 and 9 now consistently map FR1 to autonomous navigation. NFR1 specifies the hardware, language and RTOS.

9. Design Consistency

Use this section to demonstrate that the design artefacts describe one coherent system rather than independent diagrams.

Check	Student Response
Which task owns each hardware device?	Motion Control owns both motors and wheel encoders; encoder ISRs capture pulses for it to process. Line Sensor owns the three IR sensors. IMU/Terrain owns the GY-511 and I²C0. Obstacle Scan owns the HC-SR04 and SG90 servo. Wireless Comms owns WiFi/MQTT communication. The Vehicle Controller coordinates these tasks; only Motion Control writes motor PWM.
Which task has the highest priority, and why?	The Vehicle Controller has Highest priority so hazards and faults can trigger a safe stop promptly. Motion Control follows at Very high priority to maintain its ≤20 ms control cycle. Obstacle Scan is High, Line Sensor Medium, IMU/Terrain Low, and Wireless Comms Lowest. Combined-load testing must confirm the required control periods and obstacle-response time.
Which subsystem statechart corresponds to which RTOS task or subsystem?	Subsystems 1–5 correspond respectively to Wireless Comms, Motion Control, Line Sensor, IMU/Terrain and Obstacle Scan. The system-level statechart describes Vehicle Controller behaviour, including mission coordination, STOP handling and recovery. Subsystem recovery reports readiness but does not independently authorise movement.
How are functional requirements reflected in the task and statechart design?	FR1: The Vehicle Controller coordinates Line Sensor, Motion Control and Obstacle Scan to navigate the course autonomously without remote steering or human intervention. FR2–FR4: Line Sensor, Motion Control and the Vehicle Controller provide line tracking, barcode manoeuvres and encoder-based movement. FR5: IMU/Terrain detects humps and retains the highest peak. FR6: Obstacle Scan measures clearance and profiles obstacles. FR7: Obstacle Scan plans the bypass; the controller coordinates its execution and line reacquisition with Motion Control and Line Sensor. FR8: Wireless Comms handles telemetry and validated remote commands. FR9–FR10: The system-level statechart defines coordinated operation, safe stopping and controlled recovery.
What changes are still expected after the Week 6 review?	Refine task periods and priorities using timing measurements; tune control, sensing and timeout parameters; validate assumptions A1–A12; and finalise message payloads, queue capacities and overflow handling. Update affected requirements, tables and diagrams when integration evidence justifies a design change.